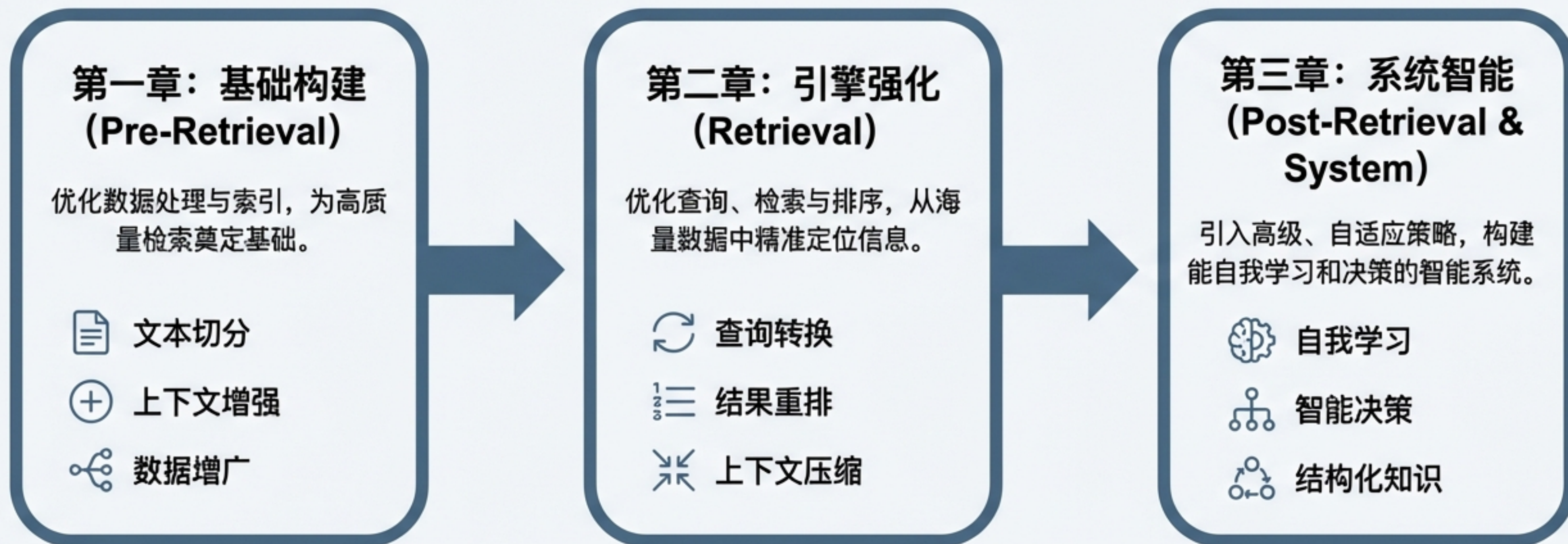


17种RAG优化策略

从基础到精通，构建顶尖问答系统的实战指南

RAG精通之路：三大核心优化领域

我们将17种策略归纳为三个核心阶段，这与一个典型的RAG系统数据流完全对应。这不仅是一个技术列表，更是一条从构建到优化的完整路径。



从高质量的文本切分开始

策略 1: Simple RAG (基础切分)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam volutpat. Ut enim ad minim veniam, quis nostrud exercitation ullamco labore es commodo commodo consequat. Duis aute inrure doloor aeprehentatal, eu se professura dolor in vugida vellura eu cñancita nulh mat laboris, ec commecindt laborum.

- **核心思想：**通过简单的文本分块和相近度匹配找到相关内容。这是最直接的策略，例如按固定大小或分隔符切分。
- **问题：**忽略了语义完整性，可能将一个完整的逻辑拆分到不同块中，导致信息丢失。

效果评分：0.3 | 实现难度：1/5

策略 2: Semantic Chunking (语义切分)

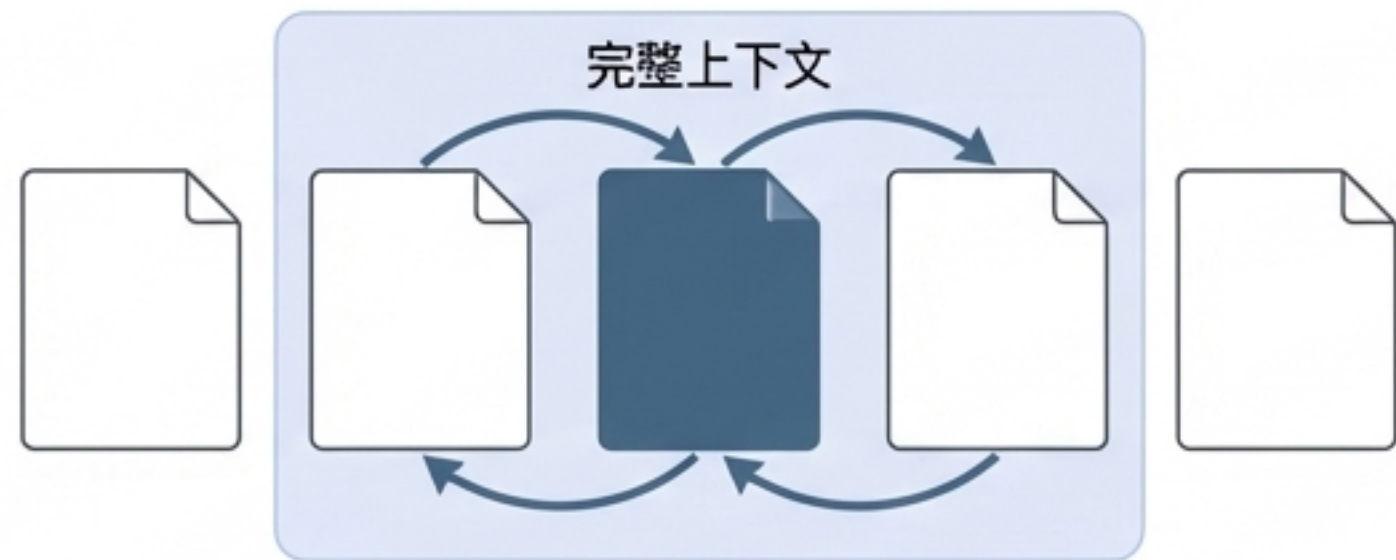
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam volutpat. Ut enim ad minim veniam, quis nostrud exercitation ullamco labore es commodo commodo consequat. Duis aute inrure doloor aeprehentatal, eu se professura dolor in vugida vellura eu cñancita nulh mat laboris, ec commecindt laborum.

- **核心思想：**基于语义的完整性来决定分块的大小，确保每个块都包含一个完整的语义单元。
- **优势：**保持了语义的连贯性，提升了单个数据块的信息质量。

效果评分：0.2 | 实现难度：2/5

丰富上下文：为检索单元注入更多信息

策略 3: Context Enriched Retrieval (上下文增强检索)

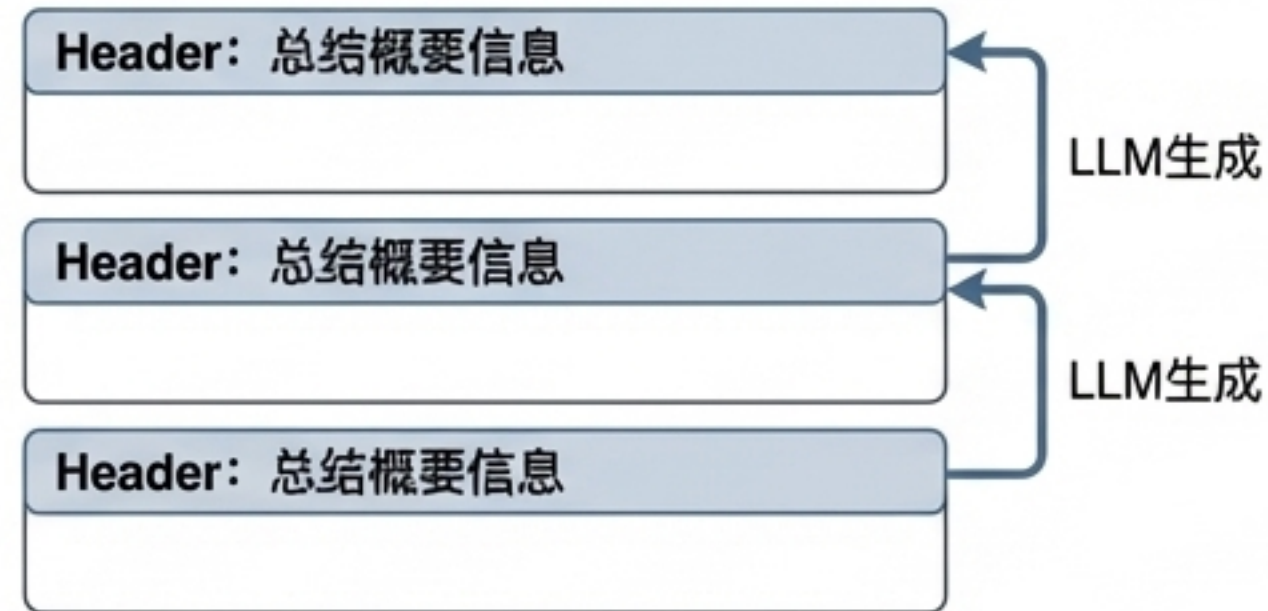


核心思想：检索到相关块时，不仅返回该块本身，还一并召回其前后相邻的部分。

问题解决：许多信息的完整理解需要依赖其上下文。该方法可以有效减少因切块导致的信息割裂。

效果评分：0.6 | 实现难度：2/5

策略 4: Contextual Chunk Headers (上下文块标题)



核心思想：为每一个数据块（chunk）动态生成一个简短、描述性的标题或摘要。

工作原理：在进行Embedding时，将块内容和其生成的标题合并。查询时，标题提供了更高级、更概括的语义信息，有助于提升检索精度。

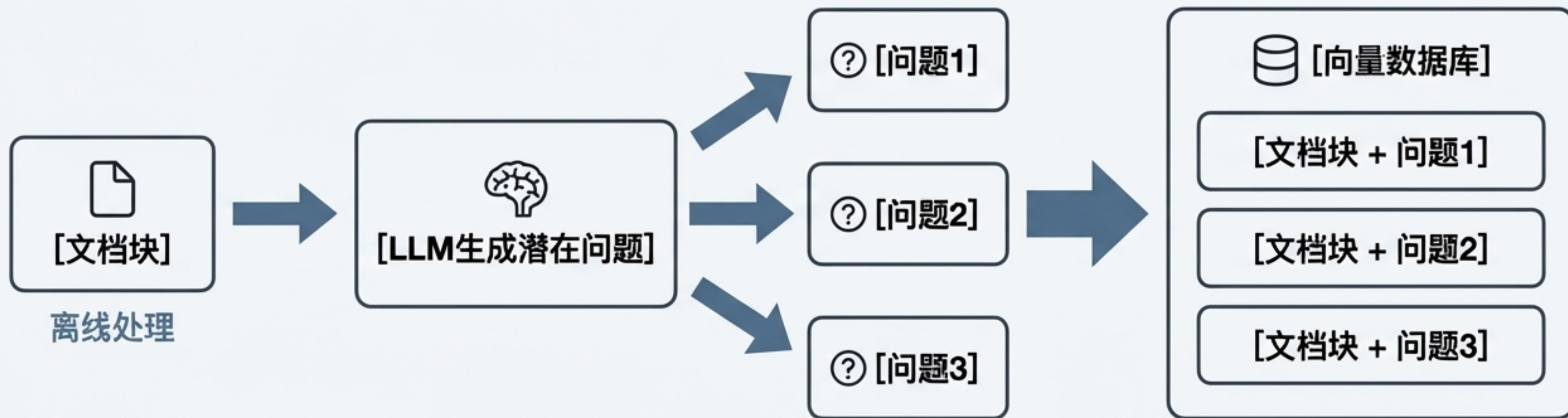
效果评分：0.5 | 实现难度：2/5

数据增广：通过生成“潜在问题”从源头提升信息密度

策略 5: Document Augmentation (文档增广)

核心思想：与其只索引“答案”（原始文本块），不如同时索引与答案匹配的“问题”。

优势：用户的查询通常更接近于“问题”的形式。通过这种方式，可以极大地提高查询和文档之间的语义相似度。



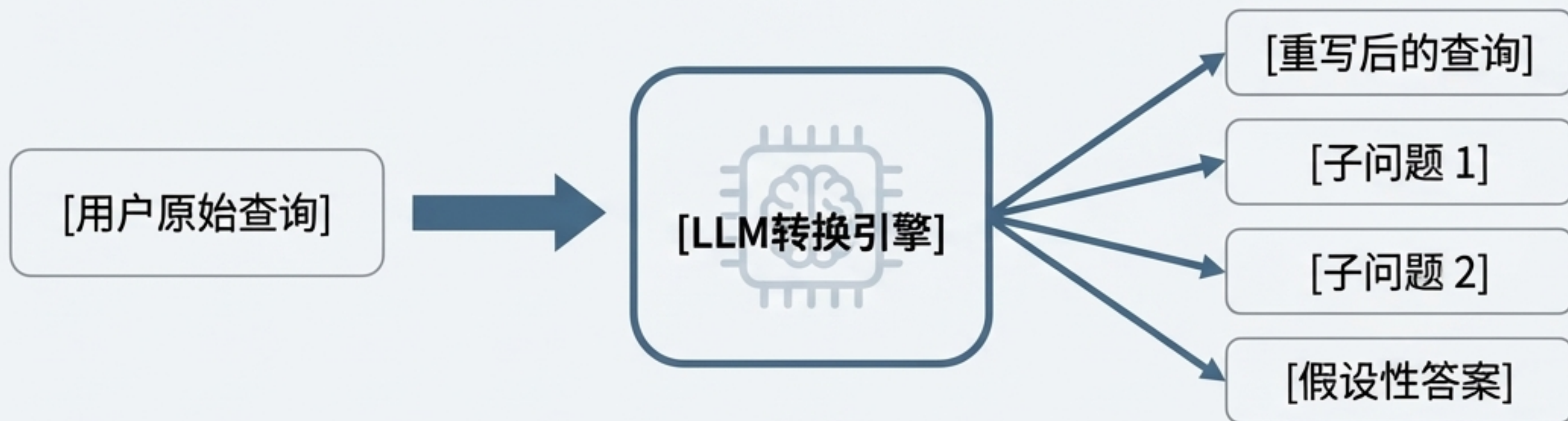
效果评分：0.8 | 实现难度：3/5

第二章：引擎强化

理解真实意图的查询转换

策略 6: Query Transformation (查询转换)

- **核心问题：**用户的原始查询可能不是最优的检索形式。它可能过于简洁、模糊或包含多种意图。
- **核心思想：**使用LLM将用户的单个查询重写或扩展为多个不同的、更适合检索的查询。



最终处理：对所有转换后查询的召回结果进行整合和排序。

效果评分：0.5 | 实现难度：3/5

精准排序：从“相似”到“最相关”

核心问题：向量相似度高的文档，不一定是业务上最相关的答案。例如，查询“苹果手机价格”，召回的文档可能都包含“苹果”，但只有一个是关于价格的。

策略 7: Reranker (重排序)

核心思想：在初步检索（召回）出一批候选文档后，使用一个更强大、更精细的模型（Reranker）进行二次排序。

优势：Reranker模型可以考虑查询和文档之间更复杂的语义关系，从而将最相关的结果排在最前面，有效过滤掉“伪相关”的噪音。

效果评分：0.7 | 实现难度：3/5

策略 8: RSE (Relevant Segment Extraction)

核心思想：进一步优化，假设最相关的信息可能分布在连续的几个文本块中，而不是单个离散的块。

实现：对初步召回的块进行上下文扩展，通过滑动窗口等方式计算连续块组合的相似度，找出最相关的连续片段。

效果评分：0.8 | 实现难度：3/5

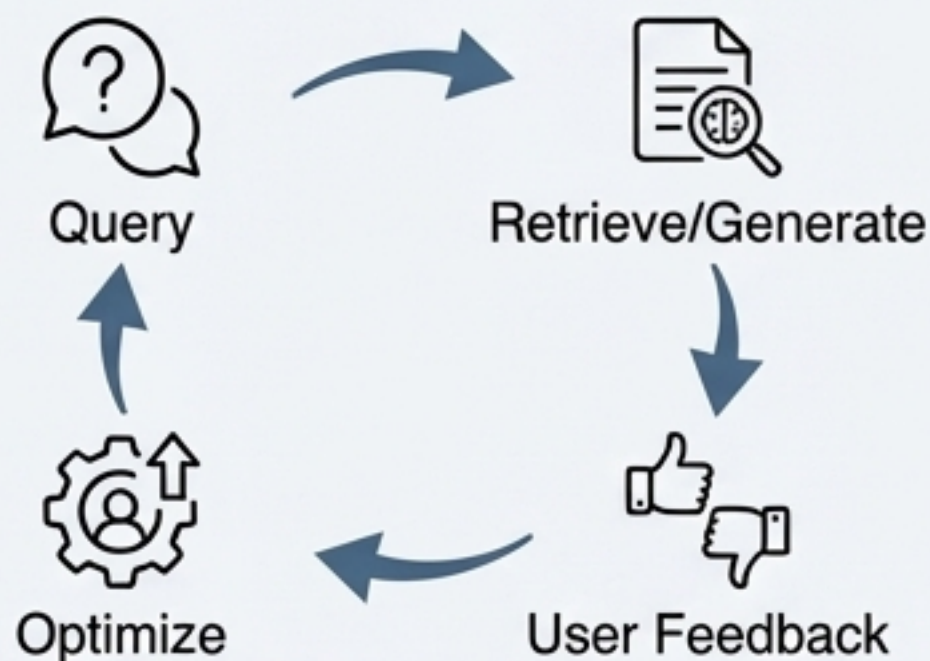


构建能自我进化的RAG

策略 9：Feedback Loop（反馈循环）

核心思想：系统不应是静态的。通过收集用户对生成答案的反馈，持续优化检索质量。

实现流程：1. 用户交互 -> 2. 收集反馈（评分/评论） -> 3. 数据存储 -> 4. 模型优化

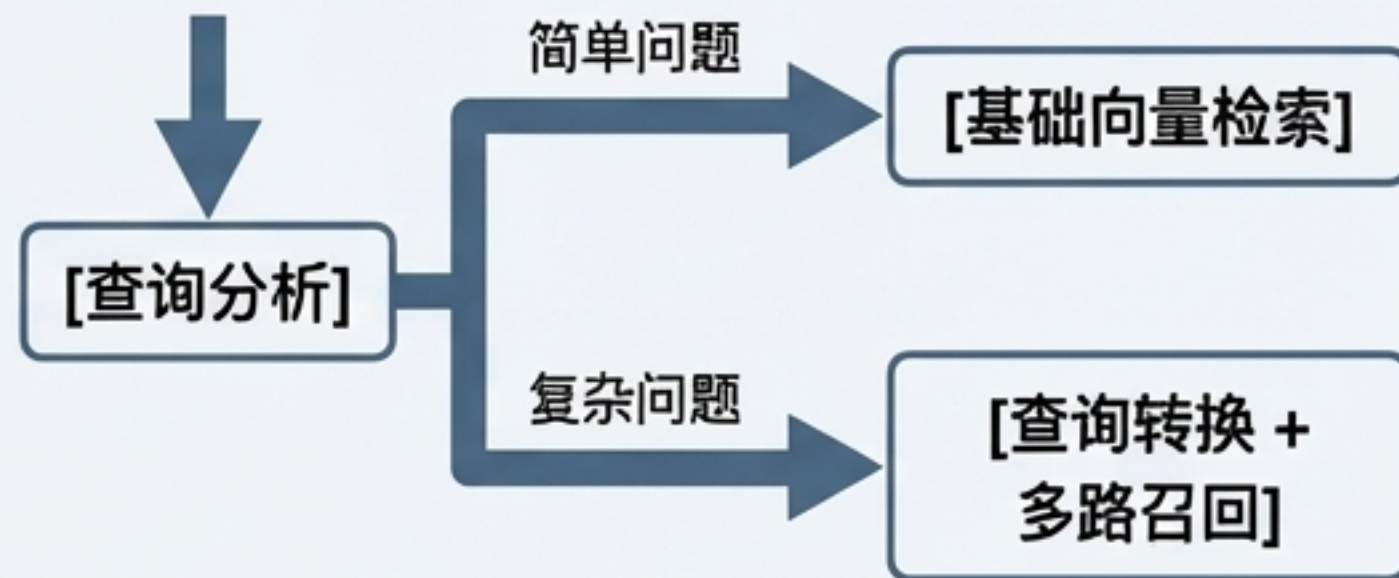


效果评分：0.7 | 实现难度：4/5

策略 10：Adaptive RAG（自适应RAG）

核心思想：识别不同类型的查询，并根据查询的复杂度和场景，动态选择最合适的检索策略。

示例：简单事实性问题使用基础检索；复杂分析性问题启动高级策略。



效果评分：0.862 | 实现难度：4/5

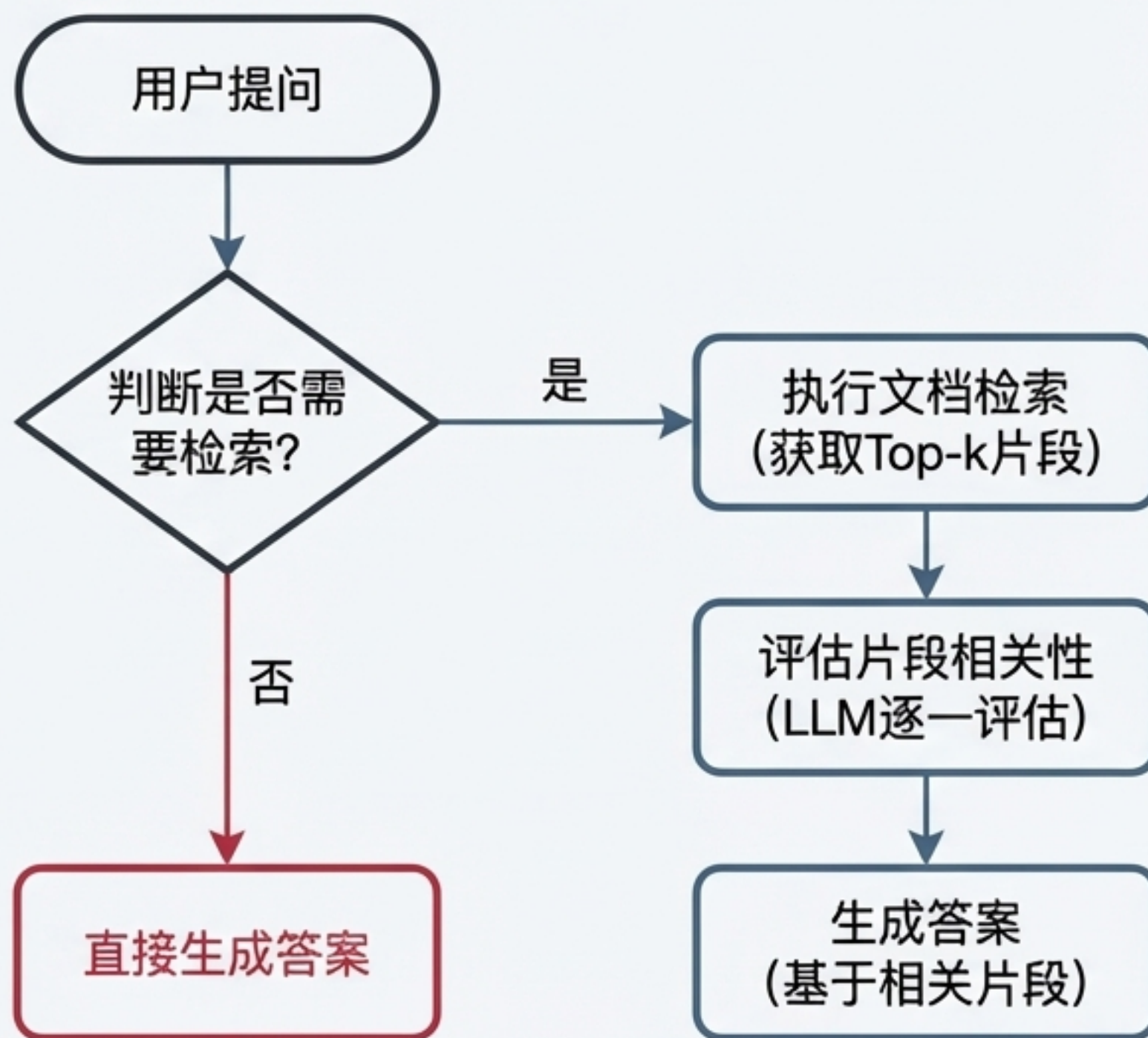
自我决策的智能体：Self RAG 详解

策略 11：Self RAG

核心思想：授予LLM更大的自主权，让它在生成答案的每一步都能自我反思和决策。

优势：减少了不必要的检索开销，并显著降低了因检索到不相关信息而导致答案“幻觉”的风险。

效果评分：0.6 | 实现难度：5/5

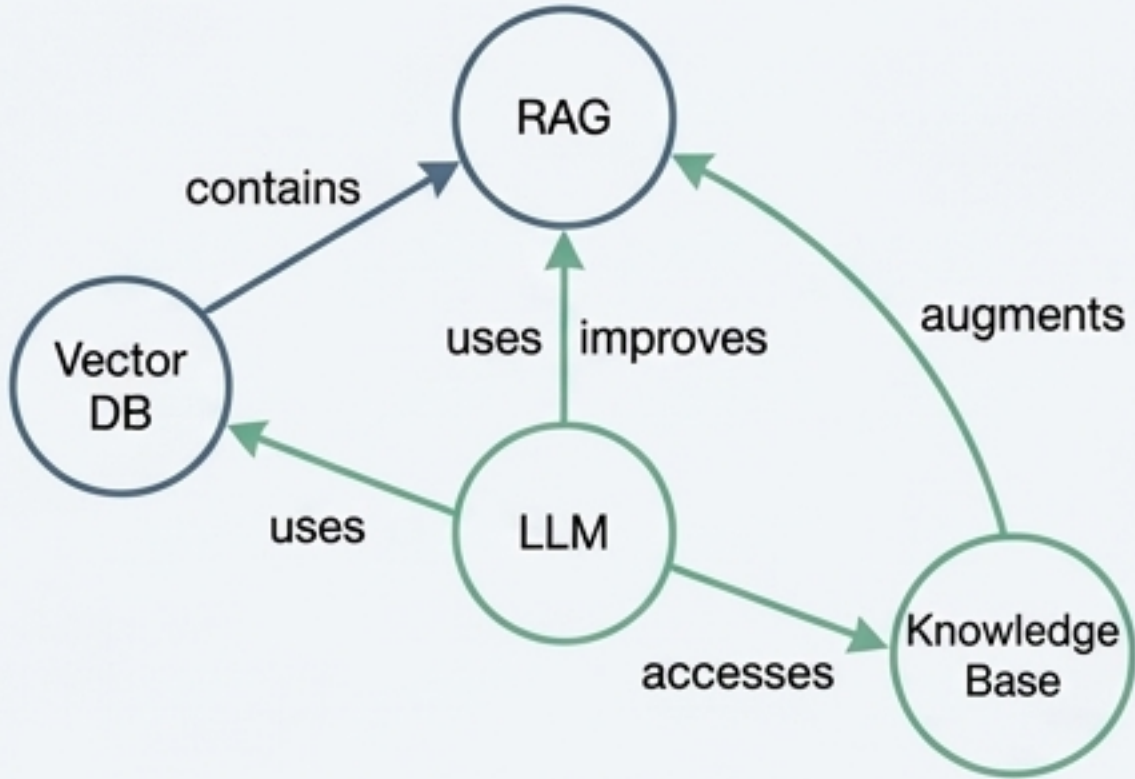


超越扁平结构：结构化知识的应用

策略 12: Knowledge Graph (知识图谱)

核心思想：将信息存储为实体（节点）和关系（边）的图结构，能更精准地表达实体间的复杂关联。

适用场景：适用于信息之间存在丰富的、明确关系时的场景。

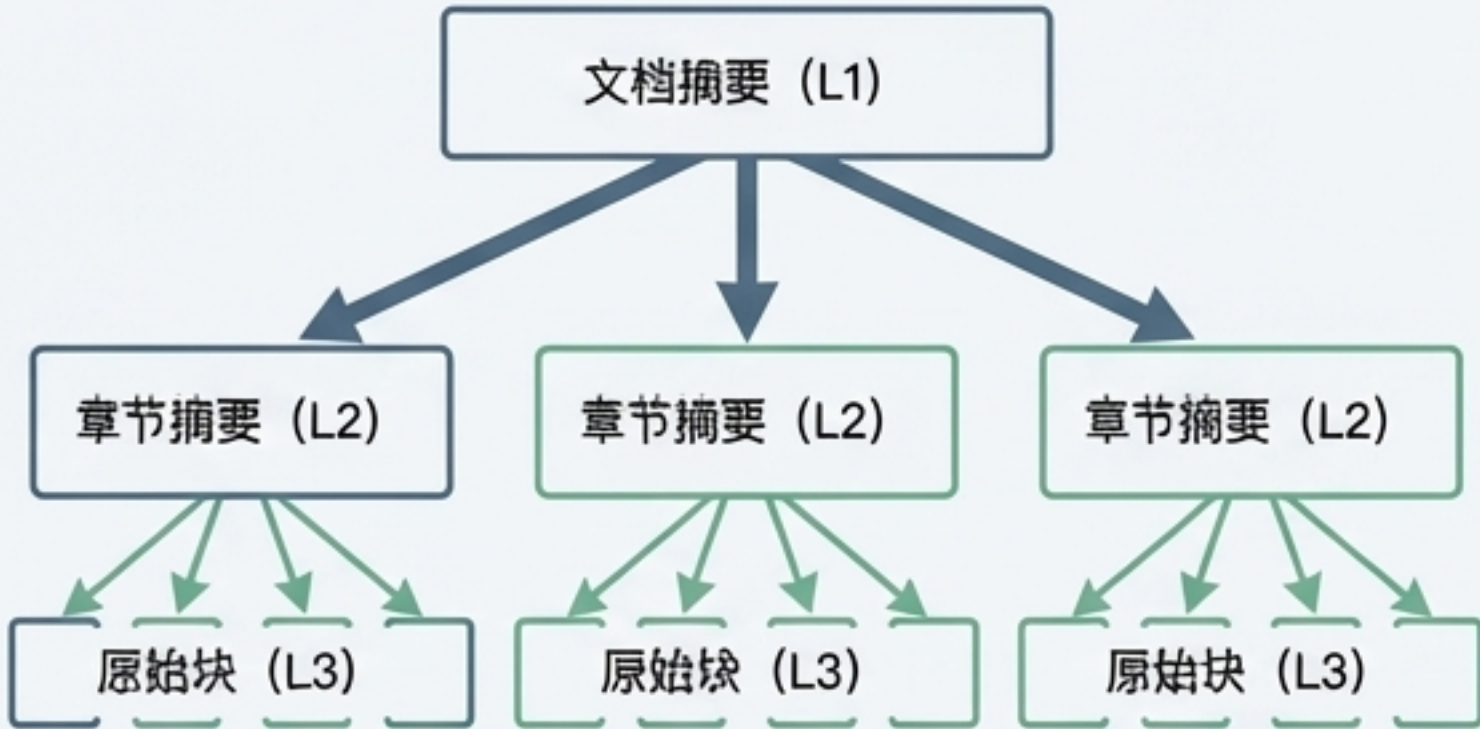


效果评分：0.78 | 实现难度：5/5

策略13: Hierarchical Indices (层级化索引)

核心思想：针对长文档，创建多层次的摘要结构，实现从宏观到微观的逐层聚焦检索。

检索流程：查询先在高层摘要中定位，再深入到相关联的下层摘要或原始块中查找。

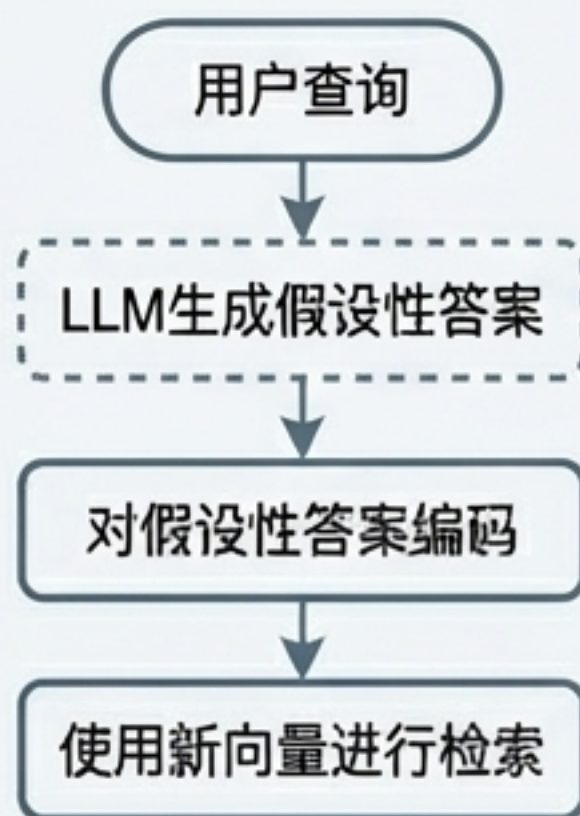


效果评分：0.84 | 实现难度：4/5

高级检索范式：HyDE 与 Fusion

策略 14: HyDE (Hypothetical Document Embeddings)

核心思想：通过“先生成、后检索”来弥合用户查询和目标答案在语义空间中的表示偏差。

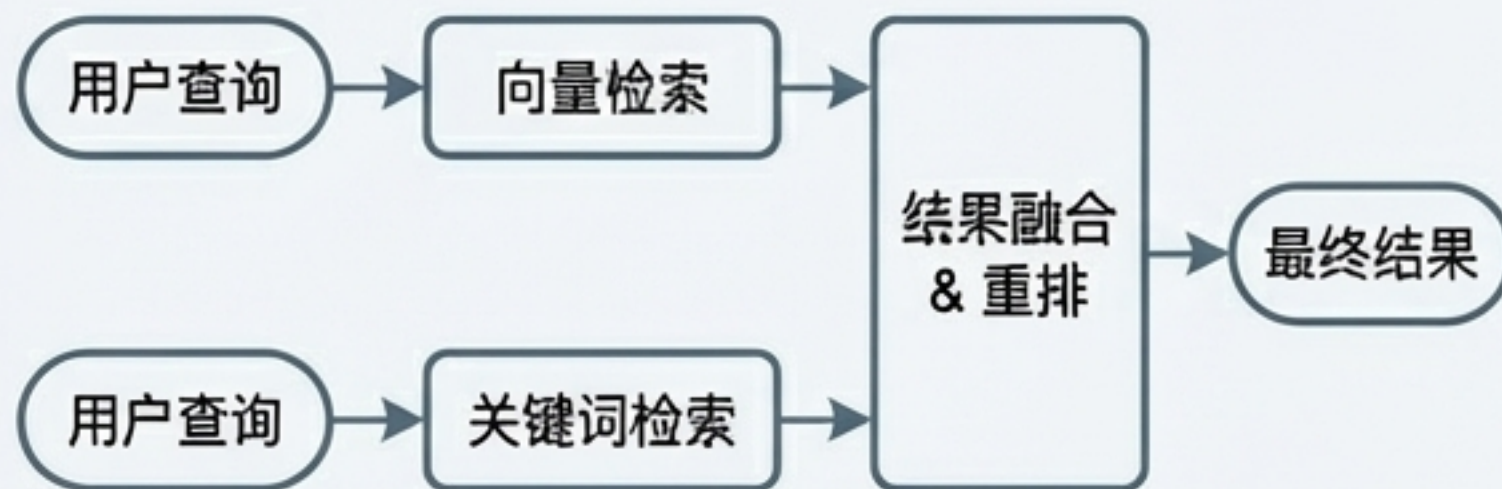


效果评分：0.5 | 实现难度：4/5

策略 15: Fusion (混合检索)

核心思想：结合多种检索方法的优势，例如关键词检索（如BM25）和向量语义检索，互为补充。

优势：语义检索理解意图，关键词检索保证精确匹配，二者结合能减少“漏检”。



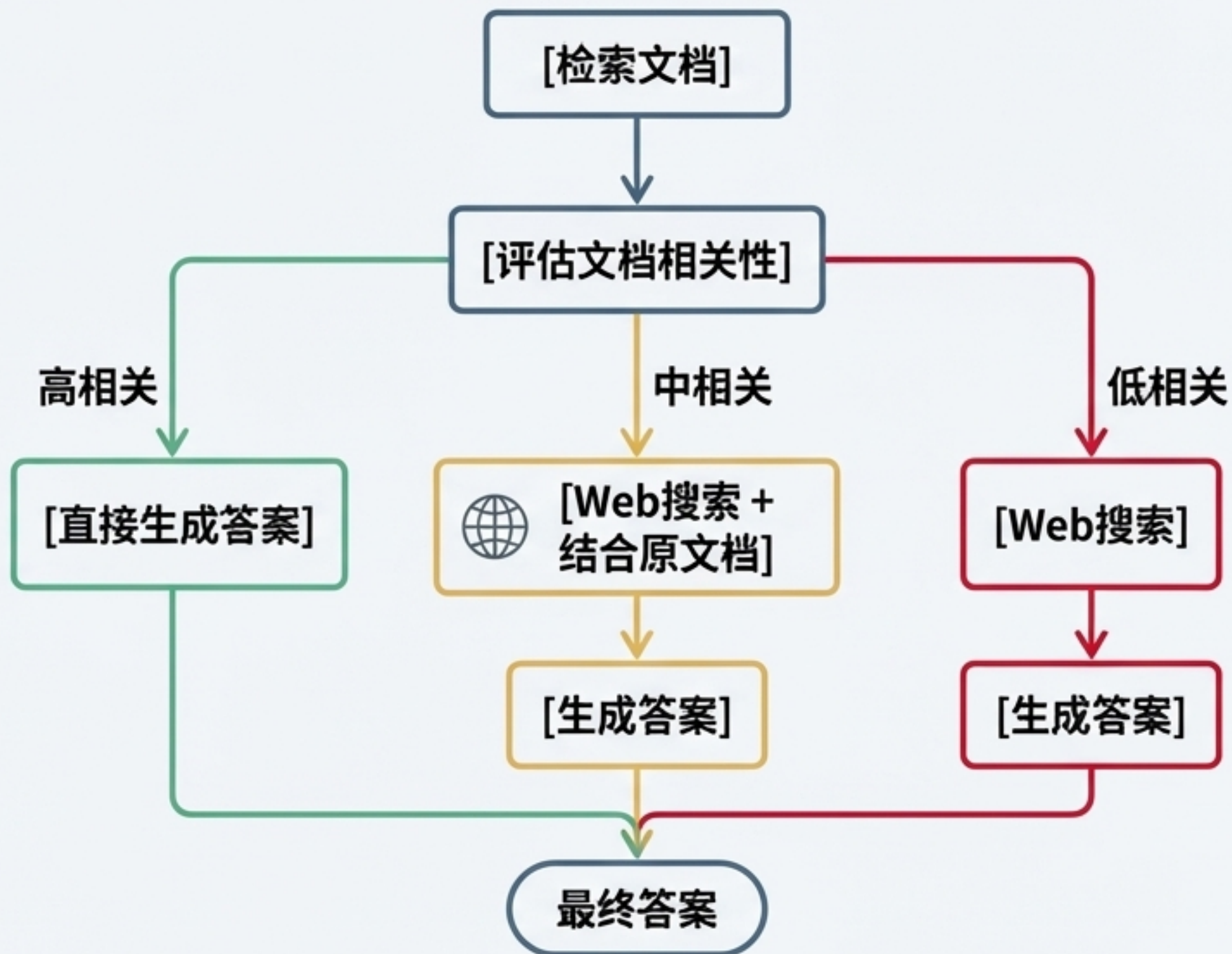
效果评分：0.83 | 实现难度：4/5

纠错与增强：CRAG 策略详解

策略 16: CRAG (Corrective-Augmented Generation)

核心思想：对检索到的文档进行严格的“事实性”评估，并基于评估结果采取不同的生成策略，从而主动纠正和增强知识。

效果评分：0.824 | 实现难度：4/5



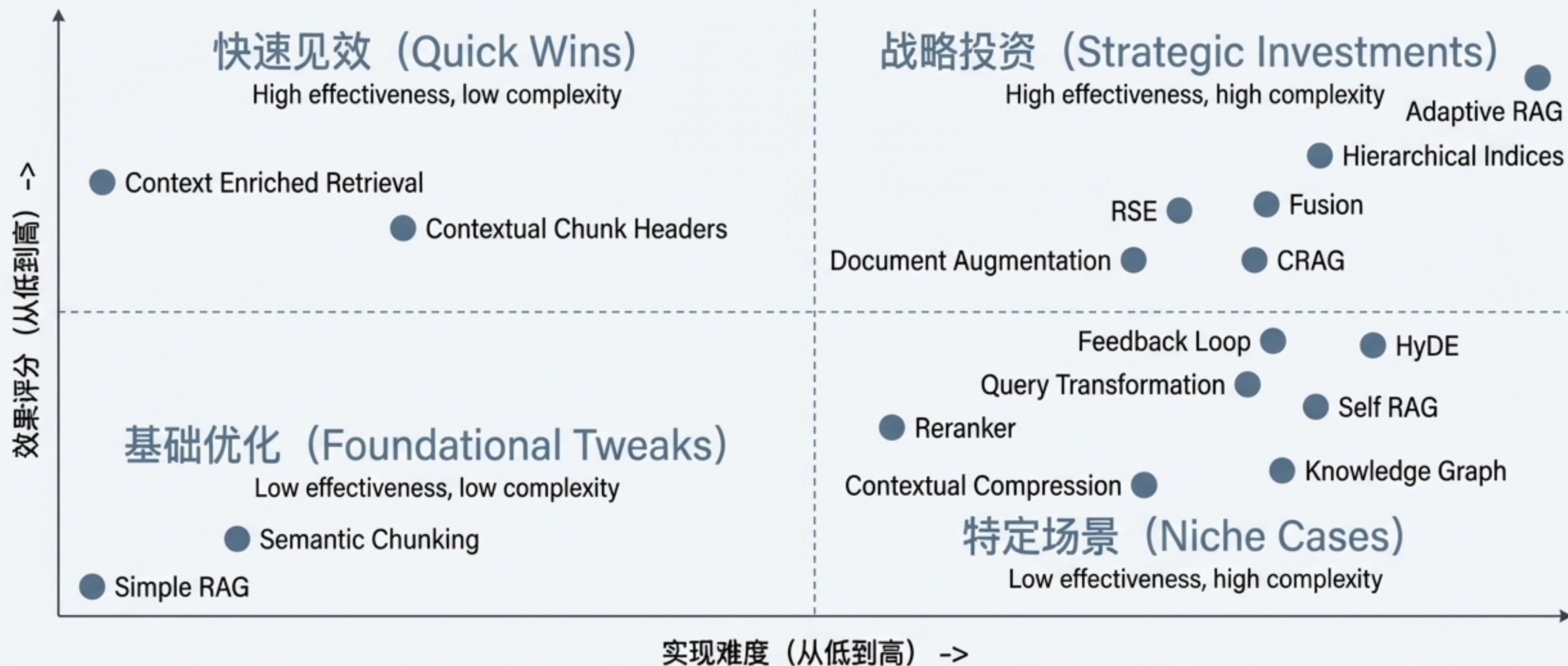
策略总览：17种RAG优化技术对比

下表系统性地总结了本次分享的所有优化策略、其核心假设/解决的问题，以及我们在内部评估的效果评分和大致的实现难度等级。

策略名称	核心假设/改进思路	效果评分	实现难度 (1-5)	一句话理由
Simple RAG	通过文本分块和相近度匹配	0.3	1	只需分块+向量检索，是所有RAG入门方案
Semantic Chunking	基于语义完整性分块	0.2	2	需借助句法/语义分析工具，确保信息完整性
Context Enriched Retrieval	检索时召回相邻上下文	0.6	2	检索时多取前后chunk，拼接即可，逻辑简单
Contextual Chunk Headers	为每个chunk添加标题/摘要	0.5	2	需自动生成标题，略有工程量，但实现难度不高
Document Augmentation	为文本块生成潜在问题作为检索入口	0.8	3	需用LLM批量生成问题，数据处理和检索逻辑更复杂
Query Transformation	将用户原始查询重写或扩展	0.5	3	需用LLM或规则对query做多种转换，需测试
Reranker	在初步检索后进行二次精排	0.7	3	需引入额外排序模型(LLM)，工程复杂度提升
RSE	找出最相关的连续文本片段	0.8	3	需实现连续块的关联性聚合，逻辑比普通检索复杂
Contextual Compression	检索后压缩上下文，剔除无关信息	0.75	3	需用LLM或规则对检索内容做摘要/压缩，需测试
Feedback Loop	利用用户反馈持续学习	0.7	4	需设计反馈收集、存储、利用机制，涉及持续学习
Adaptive RAG	根据查询类型动态选择策略	0.862	4	需做query分类和多策略切换，系统性工程
Self RAG	LLM自主决策是否、如何检索	0.6	5	需实现反思、决策、动态检索及复杂流程管理
Knowledge Graph	利用图结构表达实体关系	0.78	5	需预先构建实体关系图谱，涉及图数据库、NLP+图查询
Hierarchical Indices	创建多层吸摘要索引	0.84	4	需实现多层索引、分层检索，数据预处理和检索逻辑复杂
HyDE	先生成假设性答案再检索	0.5	4	需用LLM生成假设文档并向量化，对检索目标有假设
Fusion	融合多种检索方法（向量+关键词）	0.83	4	需维护和融合两套或多套检索系统，增加复杂性
CRAG	对检索结果进行评估并主动纠错	0.824	4	需引入评估和决策模块，并可能集成Web搜索

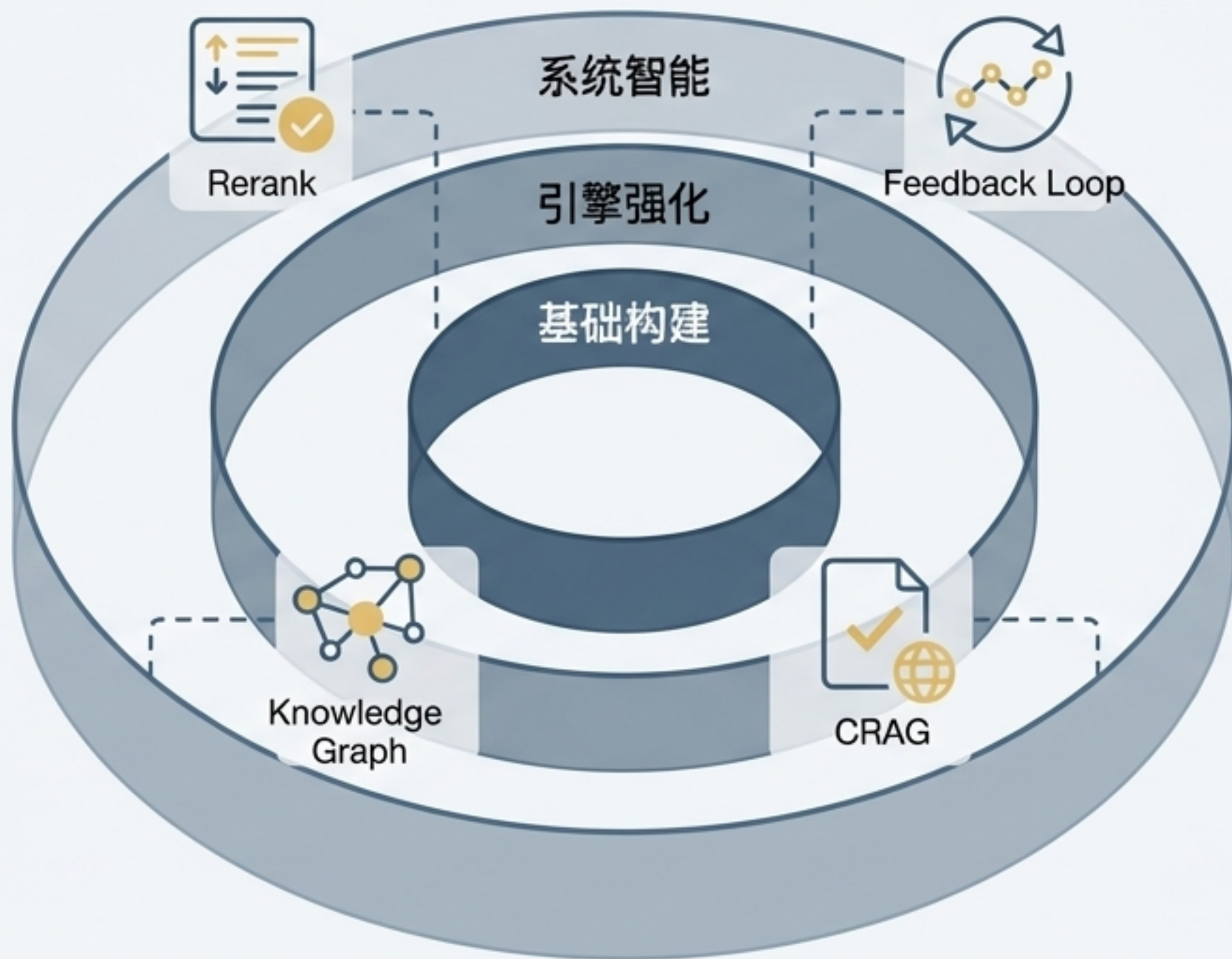
RAG策略决策矩阵：如何选择最适合你的方案？

了解所有策略后，关键问题是如何选择。这个矩阵帮助你根据“预期效果”和“投入成本”来决策，快速找到当前阶段最适合你的优化路径。



终极之道：将RAG视为一个可组合的系统

没有银弹：最优的RAG架构不是单一策略的堆砌，而是根据具体业务场景，将不同策略（如积木般）进行有机组合的结果。



度量驱动：建立反馈和评估机制，让数据驱动你的每一次优化决策。

演进式架构：从一个坚实的基础（高质量切分和索引）开始，逐步引入更高级的检索和系统智能策略。